

Hausübung 2

Worum es geht

In HÜ1 ging es nur ums Ankommen: Vue aufsetzen, erste Komponenten, ein paar Schritte machen. Unabhängig vom eigenen Kenntnisstand war das Ziel "läuft".

HÜ2 baut darauf auf. Ihr habt in Einheit 2 die drei Muster kennengelernt (Composition/Slots, Composables, Props/State/Events), und diese setzt ihr nun in einer kleinen, aber vollständigen App ein. Das Ergebnis ist deutlich umfangreicher als in HÜ1, die App bleibt jedoch bewusst klein. Der eigentliche Lerneffekt liegt darin, zu sehen, mit wie wenig Mitteln sich bereits eine saubere, wiederverwendbare Struktur erreichen lässt.

Ihr baut **QuickNotes**: eine Notiz-App mit Tags, Suche und Persistenz.

Warum das wichtig ist: Muster, die man nur erklärt bekommen hat, sitzen nicht — erst beim eigenständigen Bauen merkt ihr, wo Composition, Composables oder Ein-Weg-Datenfluss euch wirklich Arbeit ersparen.

Das AI-Thema (bitte vor Beginn lesen)

Ihr dürft und sollt mit AI arbeiten. Das ist ausdrücklich Teil der Übung. Die Regel lautet jedoch: **Ihr müsst verstehen, was ihr abgebt.**

Konkret bedeutet das:

- Nutzt AI zum Erklären, Generieren und Aufräumen, so wie ihr es braucht.
- Wenn eine AI euch etwas ausgibt, das ihr nicht erklären könnt, habt ihr zwei Möglichkeiten: es verstehen oder es entfernen. Bitte nichts ungeprüft übernehmen.
- Ihr führt ein `AI-LOG.md` (siehe unten). Dieses ist kein Kontrollinstrument, sondern euer Nachweis, dass ihr bewusst gearbeitet habt.
Im Peer-Review und in der Prüfung werden Fragen zu eurem eigenen Code gestellt. Wer den Code lediglich durchgereicht hat, wird dies dort merken.

Was die App können muss

Alle folgenden Punkte sind Pflicht und für alle gleich:

1. **Notizen anlegen** mit Titel, Textinhalt und beliebig vielen Tags.
2. **Notizen löschen.**
3. **Live-Suche / Filter:** Während der Eingabe wird die Liste nach Titel, Text oder Tag gefiltert.
4. **Persistenz:** Nach einem Reload der Seite sind die Notizen weiterhin vorhanden (localStorage).
Das ist der vollständige Funktionsumfang, nicht mehr. Wer früher fertig ist, geht in den optionalen Bonus (ganz unten), nicht in neue Features.

Vorgegebene Struktur

Diese Dateistruktur ist verbindlich. Dafür gibt es zwei Gründe: Erstens erzwingt sie, dass ihr alle drei Muster aus Einheit 2 einsetzt. Zweitens weiß im Peer-Review jede Person sofort, wo nachzuschauen ist, andernfalls ist die Bewertung in 30 Minuten nicht zu schaffen.

```
src/  
  App.vue  
  components/  
    BaseCard.vue      ← Slots (Thema A)  
    NoteCard.vue      ← nutzt BaseCard  
    NoteForm.vue      ← neue Notiz eingeben  
    SearchBar.vue      ← v-model + emit (Thema C)  
  composables/  
    useNotes.js        ← Notiz-Logik (Thema B)  
    useLocalStorage.js ← Persistenz (Thema B)  
  types/  
    note.ts           ← das Note-Interface (TypeScript)
```

Ihr dürft zusätzliche Dateien anlegen, diese müssen jedoch vorhanden sein und ihre Rolle erfüllen.

Was in welcher Datei passieren soll

composables/useNotes.js kapselt die gesamte Notiz-Logik: die Liste der Notizen, `addNote()`, `deleteNote()` sowie eine gefilterte Ansicht. Die Komponenten rufen nur diese Funktionen auf, sie verwalten die Liste nicht selbst.

composables/useLocalStorage.js übernimmt das Speichern und Laden. Wichtig: Die `localStorage`-Zugriffe gehören hierher, nicht verstreut in die Komponenten. **useNotes** verwendet **useLocalStorage**.

components/BaseCard.vue ist eine generische Card mit einem benannten `#header`-Slot und einem Default-Slot. Sie weiß nichts von Notizen. Genau das ist der Kern (Thema A).

components/NoteCard.vue rendert eine einzelne Notiz und verwendet dafür **BaseCard**. Der Löschen-Button meldet per `emit` nach oben, er löscht nicht selbst.

components/SearchBar.vue ist eine eigene Input-Komponente mit `defineProps(['modelValue'])` und `emit('update:modelValue', ...)`, sodass sie sich mit `v-model` einbinden lässt (Thema C).

components/NoteForm.vue nimmt eine neue Notiz auf und meldet diese per `emit` an **App.vue**.

Ihr müsst nicht bei null beginnen: in Moodle findet ihr ein kleines Gerüst.

TypeScript

TypeScript soll hier eingesetzt werden. Definiert mindestens das `Note`-Interface in `types/note.ts` und verwendet es.

Ein aufrichtiger Hinweis dazu: Arbeitet so, wie ihr gut vorankommt. Wenn TypeScript euch an einer Stelle mehr aufhält als es hilft, ist ein sauber laufendes Ergebnis mit weniger TS mir lieber als eine perfekt typisierte App, die ihr nicht fertigstellt. Verstehen und abgeben geht vor Perfektion.

Trotzdem der klare Hinweis: In der Prüfung gibt es eine Frage zu TypeScript in diesem Projekt (warum ein `Note`-Interface, was es beim Datenfluss zwischen den Komponenten bringt). Wer TS ganz weggelassen hat, kann diese eine Frage schlechter beantworten und verliert dort einen Punkt. Das ist bewusst so angelegt, damit sich der Aufwand lohnt, ohne euch beim Coden zu blockieren.

Das AI-LOG

Legt eine Datei `AI-LOG.md` im Repository an. Pro nennenswertem AI-Einsatz eine kurze Zeile, kein Fließtext:

```
- Prompt: "wie mache ich in Vue ein v-model auf einer eigenen Komponente"  
  Übernommen: das emit-Muster in SearchBar  
  Geändert/verstanden: Prop heißt bei mir searchTerm, verstanden warum emit statt Prop-Mutation
```

Drei bis acht solche Einträge sind völlig ausreichend. Es geht nicht um Vollständigkeit, sondern darum, dass ihr nachvollziehen könnt, wo AI euch geholfen hat und was ihr selbst verantwortet.

README

Eure `README.md` enthält:

1. **Setup-Anleitung** (wie startet man das Projekt).
2. **Kurze Begründung eurer Struktur** (zwei, drei Sätze: warum liegt die Logik im Composable und nicht in der Komponente).
3. **Drei Reflexionsfragen**, zum Code-Verständnis kurz beantwortet:
 - Warum darf `NoteCard` die `Notiz-Prop` nicht selbst verändern, und wie löst ihr das stattdessen?
 - Was passiert, wenn zwei Komponenten dasselbe `useNotes()` aufrufen — teilen sie sich die Notizen oder nicht? Begründet kurz.
 - Wozu dient das `Note`-Interface, wenn der Code auch ohne `liefere`?

Bewertung

Nicht die Abgabe selbst wird benotet, sondern das Code-Review dazu in der darauffolgenden Einheit. Die folgenden Kriterien sind, worauf beim Review geachtet wird — bereitet euren Code entsprechend auf.

Bereich	Worauf geachtet wird
Funktioniert	Alle vier Pflichtfunktionen laufen, Reload behält die Notizen
Struktur	Vorgegebene Dateien vorhanden und in ihrer Rolle genutzt
Thema A	<code>BaseCard</code> mit benanntem Slot, von <code>NoteCard</code> verwendet
Thema B	Logik und Persistenz liegen in Composables, nicht in Komponenten
Thema C	Ein-Weg-Datenfluss, Löschen und Suche laufen über <code>emit</code> bzw. <code>v-model</code>
Verständnis	README-Reflexion und AI-LOG plausibel, Prüfungsfragen beantwortbar
TypeScript	<code>Note</code> -Interface vorhanden und genutzt (eine Prüfungsfrage hängt daran)

Abgabe

- Format: Link zu eurem GitHub-Repository
- Upload in Moodle (Bezeichnung dort prüfen)
- Abgabedeadline siehe Moodle